

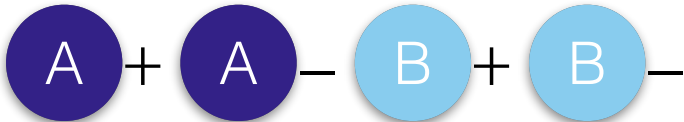
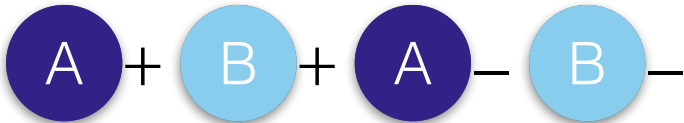






Perminituted

upd(+ -), log(+ -)

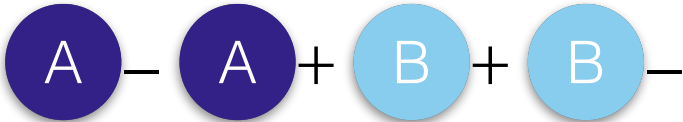


order

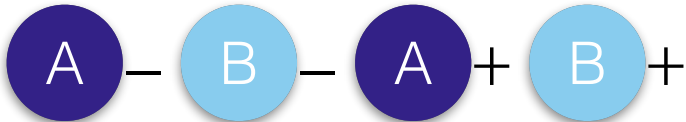
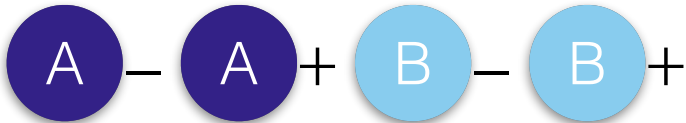
$\text{upd}(+-), \log(-+)$



upd(-+), log(+ -)



upd(-+), log(-+)











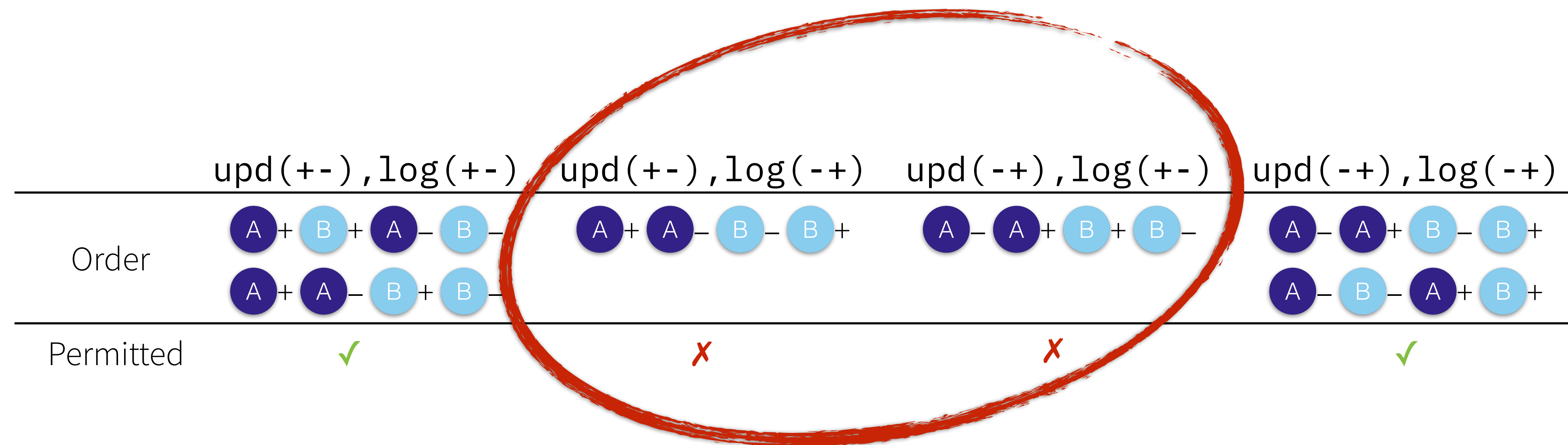


But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) {    
    account.balance += amount  
    val id = account.id()  
    when(log) {  log.record(id, amount) }  
  }  
}
```

But some orders are the point!

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    account.balance += amount  
    val id = account.id()  
    when(log) { B log.record(id, amount) }  
  }  
}
```





Ordering Design Insights

Intrinsic — Program structure determines which orders are allowed

Incidental — Scheduling and runtime must not